

Algoritmos Genéticos

Fundamentos teóricos y diseño de algoritmos evolutivos



Inspiración Biológica

Los algoritmos genéticos pertenecen a la familia de los algoritmos evolutivos, inspirándose en la teoría de la evolución por selección natural propuesta por Charles Darwin. En la naturaleza, las especies evolucionan mediante un proceso gradual basado en tres principios fundamentales que guían la adaptación y supervivencia.



Variabilidad

Los individuos de una población presentan diferencias entre sí que afectan su capacidad para sobrevivir y reproducirse.



Selección Natural

El entorno favorece a los individuos mejor adaptados, con mayor probabilidad de transmitir sus características.



Herencia y Mutación

Los descendientes heredan rasgos de sus progenitores, pero con pequeñas variaciones que introducen diversidad.

Del Mundo Natural al Computacional

Los algoritmos genéticos trasladan los conceptos evolutivos al ámbito computacional. La idea clave es que no se busca la mejor solución de forma directa, sino que se evoluciona una población de soluciones hacia soluciones cada vez mejores.

Biología	Algoritmos Genéticos
Individuo	Solución candidata
Población	Conjunto de soluciones
Gen	Parte de la solución
Fitness	Calidad de la solución
Mutación	Cambio aleatorio
Evolución	Mejora progresiva

Flujo General de un Algoritmo Evolutivo

Aunque existen muchas variantes, todos los algoritmos evolutivos siguen el mismo esquema general. Este proceso iterativo permite la mejora progresiva de las soluciones a través de múltiples generaciones.



Inicialización

Se genera una población inicial de soluciones, normalmente de forma aleatoria. La diversidad inicial es clave para evitar soluciones pobres.



Evaluación (Fitness)

Cada solución se evalúa mediante una función de fitness que mide su calidad. La definición del fitness depende exclusivamente del problema.



Selección

Se seleccionan individuos para reproducirse. Las soluciones mejores tienen mayor probabilidad, pero se mantiene cierto azar para preservar diversidad.



Cruce

Se combinan soluciones para generar nuevas. El cruce permite reutilizar información útil de soluciones previas.



Mutación

Se introducen pequeños cambios aleatorios. La mutación evita la convergencia prematura y permite explorar nuevas soluciones.



Reemplazo

Se decide qué individuos forman la siguiente generación. Esta decisión influye enormemente en estabilidad y convergencia.

Criterios de Parada

El algoritmo se detiene cuando se cumple alguna de las siguientes condiciones. Elegir el criterio adecuado es fundamental para equilibrar calidad de solución y tiempo de ejecución.

1

Máximo de Generaciones

Se alcanza un número predefinido de iteraciones del algoritmo evolutivo.

2

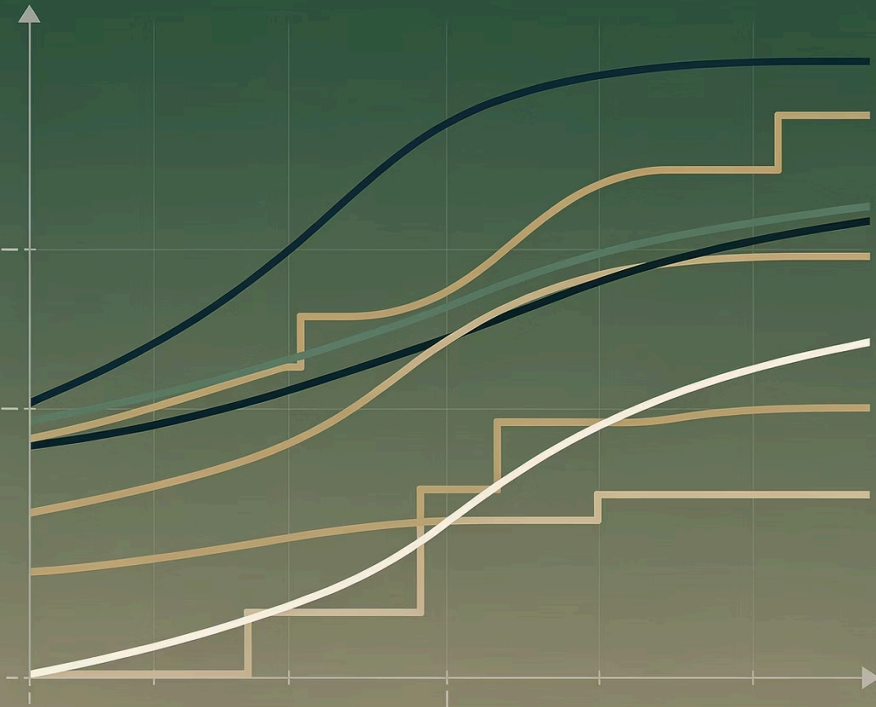
Solución Suficientemente Buena

Se obtiene una solución que cumple con el umbral de calidad establecido.

3

Estancamiento

No hay mejoras significativas en la población durante varias generaciones consecutivas.

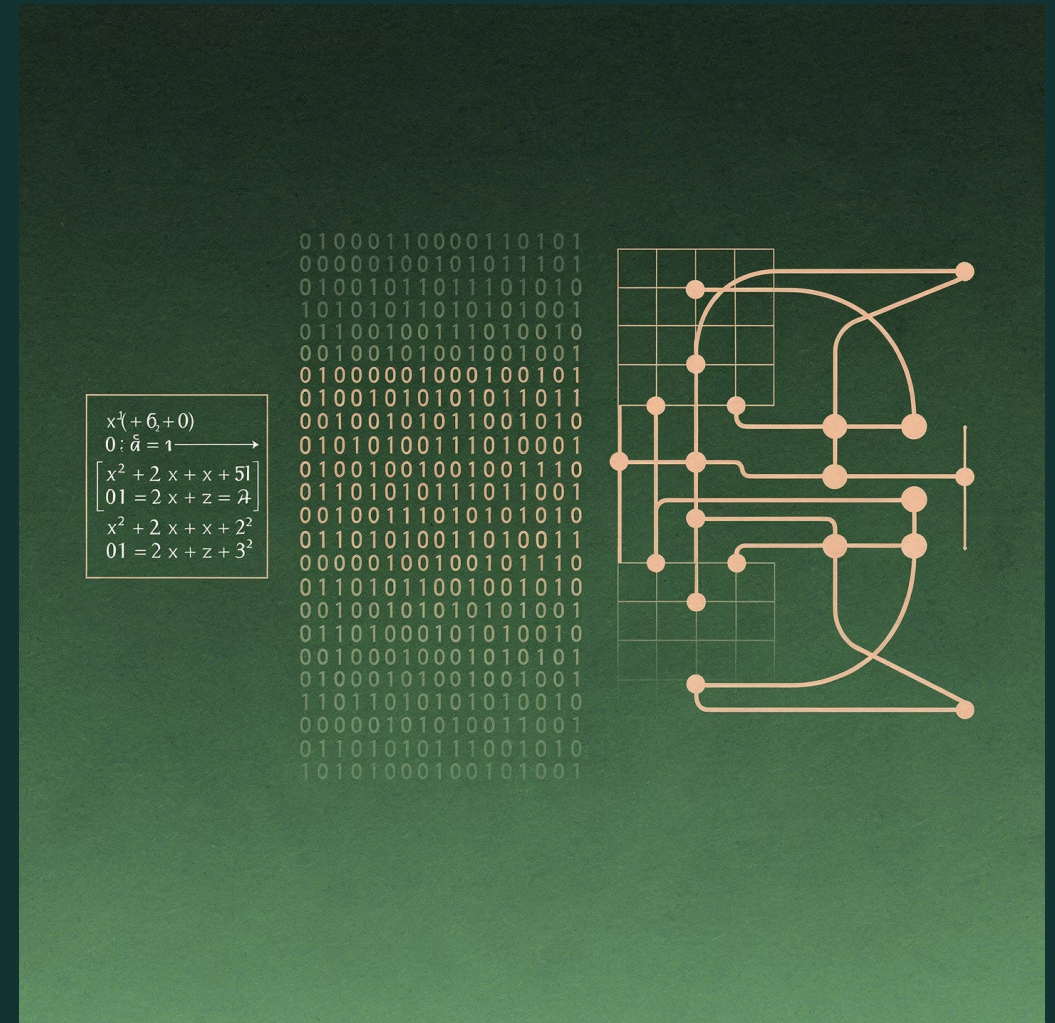


Algoritmos Genéticos Simples

Representación de Soluciones

La representación define cómo se codifica una solución. Debe ser válida, coherente con el problema y compatible con los operadores genéticos.

- Binaria: cadenas de 0s y 1s
- Valores reales: números continuos
- Permutaciones: ordenaciones de elementos



Función de Fitness

La función de fitness guía la evolución y define qué significa "mejor solución". Puede incluir penalizaciones para soluciones inválidas. Una mala función de fitness produce malos resultados, aunque el algoritmo esté bien implementado.

Operadores Genéticos: Selección

La selección determina qué individuos se reproducen. Existe un compromiso fundamental entre presión selectiva y mantenimiento de diversidad en la población.

Ruleta

Probabilidad proporcional al fitness. Permite que todos participen pero favorece a los mejores.

Torneo

Competición entre subgrupos aleatorios. Fácil de implementar y controlar la presión selectiva.

Ranking

Selección basada en posición ordenada. Reduce el impacto de diferencias extremas en fitness.



Operadores Genéticos: Cruce y Mutación

Cruce

El cruce depende de la representación elegida:

- Cruce de un punto
- Dos puntos
- Uniforme
- OX (Order Crossover), PMX (Partially Mapped Crossover) para permutaciones

Mutación

La mutación introduce exploración y evita estancamiento. Suele aplicarse con baja probabilidad y debe respetar la validez de la solución.

Permite escapar de óptimos locales y mantener la diversidad genética a lo largo de las generaciones.

📌 **Importante:** Un cruce mal diseñado puede generar soluciones inválidas.

Aplicaciones y Estrategias de Reemplazo

Aplicaciones Reales

Los AG son especialmente útiles en:

- Problemas combinatorios
- Espacios de búsqueda grandes
- Problemas sin solución analítica

Ejemplos: problema del viajante, asignación de tareas, planificación, selección de características.

Estrategias de Reemplazo

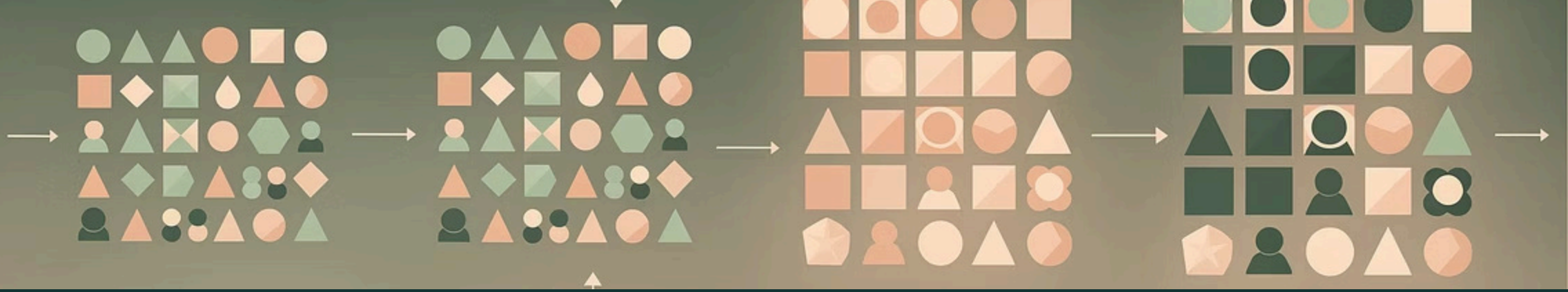
Generacional completo: Toda la población es reemplazada. Simple pero inestable.

Elitismo: Se conserva el mejor individuo. Introduce memoria evolutiva.

Modelo ($\mu + \lambda$): Padres e hijos compiten. Muy estable y muy usado.

Steady-State: La población cambia lentamente. Mantiene diversidad.

No existe una estrategia universalmente mejor. La elección depende del problema específico y los objetivos del algoritmo.



💡 CONCLUSIÓN

Diseño y Conclusiones

Los algoritmos genéticos son una herramienta potente para resolver problemas complejos. Su eficacia depende más del diseño que del código.

01

Elegir representación

Definir cómo se codifican las soluciones

03

Seleccionar operadores

Elegir métodos de selección, cruce y mutación

02

Definir fitness

Establecer la función de evaluación

04

Decidir reemplazo

Determinar la estrategia generacional

No existen recetas universales. El diseño depende del problema. Comprender los principios teóricos permite adaptar el algoritmo, interpretar resultados y tomar decisiones fundamentadas.